



DOSSIER DE PROJET

Trouve ton artisan

Plateforme de mise en relation avec les artisans
de la Région Auvergne-Rhône-Alpes

Formation / évaluation	Projet Web full-stack
Nom / prénom	Clément Michenaud
Date	Septembre 2026
Technologies	ReactJS, Bootstrap, Sass, Node.js, Express, Sequelize, MySQL/MariaDB

Liens du projet

Les maquettes Figma, le repository GitHub et le site en ligne sont disponibles et renseignés dans le dossier final.

Sommaire

1. Contexte et expression du besoin
2. Contraintes et livrables attendus
3. Maquettes et identité graphique
4. Architecture technique
5. Base de données - MCD et MLD
6. Sécurité mise en place
7. Veille sécurité et vulnérabilités
8. Accessibilité, responsive et SEO
9. Repository, installation et scripts SQL
10. Hébergement et mise en ligne
11. Conclusion et checklist de remise

Périmètre du dossier

Ce document reprend les exigences du brief et décrit la solution fournie dans le dossier de code joint. Les données artisans proviennent du fichier Excel fourni avec le sujet.

1. Contexte et expression du besoin

1.1 Présentation du commanditaire

La Région Auvergne-Rhône-Alpes souhaite proposer une plateforme dédiée aux artisans de son territoire. Le brief indique que la région couvre douze départements et possède notamment des bureaux à Lyon et Clermont-Ferrand. Le projet est mené avec l'antenne lyonnaise.

Le contexte métier justifie la création de la plateforme : l'artisanat représente une part importante du tissu économique régional et le commanditaire souhaite faciliter la mise en relation entre particuliers et professionnels.

1.2 Besoin utilisateur

L'objectif fonctionnel est de permettre à un particulier de trouver rapidement un artisan puis de lui demander des renseignements, une prestation ou un tarif grâce à un formulaire de contact.

- Naviguer dans les quatre catégories : Bâtiment, Services, Fabrication et Alimentation.
- Rechercher un artisan par son nom.
- Consulter une liste d'artisans puis une fiche détaillée.
- Voir les trois artisans du mois sur la page d'accueil.
- Envoyer une demande par formulaire sans afficher publiquement l'adresse e-mail de l'artisan.
- Obtenir une interface simple sur mobile, tablette et ordinateur.

1.3 Données métier

Le jeu d'essai contient 17 artisans. Chaque artisan appartient à une seule spécialité et chaque spécialité dépend d'une seule catégorie. Ce principe est conservé dans le modèle relationnel afin d'éviter la duplication de la catégorie dans chaque artisan.

Catégorie	Nombre d'artisans
Alimentation	4
Bâtiment	4
Fabrication	3
Services	6
Artisans du mois	3

2. Contraintes et livrables attendus

2.1 Contraintes imposées

Contrainte	Mise en œuvre retenue
Technologies front-end	ReactJS, Bootstrap et Sass.
API et données	Node.js, Express, Sequelize et MySQL ou MariaDB.
Maquettage	Figma, avec déclinaisons mobile, tablette et ordinateur.
Responsive	Approche mobile first.
Accessibilité	Conformité visée avec WCAG 2.1 et interface utilisable par le public le plus large possible.
Sécurité	Application durcie selon les bonnes pratiques Web ; la base de données n'est jamais accessible directement depuis le navigateur.
Qualité	Code propre, commenté, correctement indenté et vérifiable via les outils W3C.
Hébergement	Le site doit être accessible en ligne.
Versionnement	Git et GitHub.

2.2 Pages et fonctionnalités attendues

- Header commun avec logo cliquable, catégories alimentées depuis la base et barre de recherche.
- Footer commun avec pages légales et coordonnées de l'antenne de Lyon.
- Accueil : rubrique « Comment trouver mon artisan ? » en quatre étapes et trois artisans du mois.
- Liste d'artisans par catégorie ou recherche, sous forme de cartes cliquables.
- Fiche artisan : nom, visuel, note, spécialité, ville, à propos, formulaire de contact et site web éventuel.
- Pages légales temporaires affichant un contenu « Page en construction ».
- Page 404 pour toutes les routes inconnues.
- Titre et description SEO pour chaque page.

Adresse du footer

101 cours Charlemagne - CS 20033 - 69269 Lyon Cedex 02 - France - +33 (0)4 26 73 40 00.

3. Maquettes et identité graphique

3.1 Direction visuelle

La solution reprend la palette fournie dans le brief : bleu principal #0074c7, bleu profond #00497c, anthracite #384050, rouge #cd2c2e, vert #82b864 et fond clair #f1f8fc. La police prévue est Graphik avec une pile de polices de secours dans le code.

Police Graphik

Les fichiers de police ne sont pas redistribués dans le projet. Le CSS référence Graphik en premier choix ; il faut ajouter localement les fichiers autorisés par votre licence.

3.2 Maquettes responsive

Les captures ci-dessous constituent les maquettes de référence à reproduire ou importer dans Figma. Les écrans couvrent les principaux gabarits demandés : accueil, liste/recherche, fiche artisan, pages légales et 404.

Accueil - déclinaisons responsive

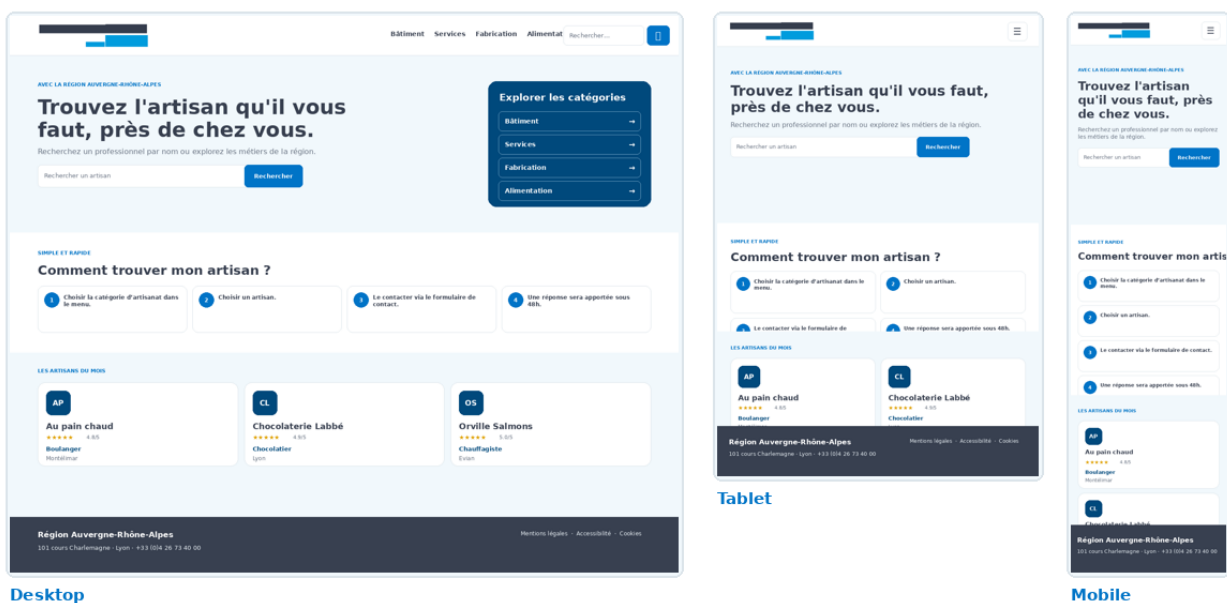


Figure 1 - Accueil : ordinateur, tablette et mobile.

Liste d'artisans - déclinaisons responsive

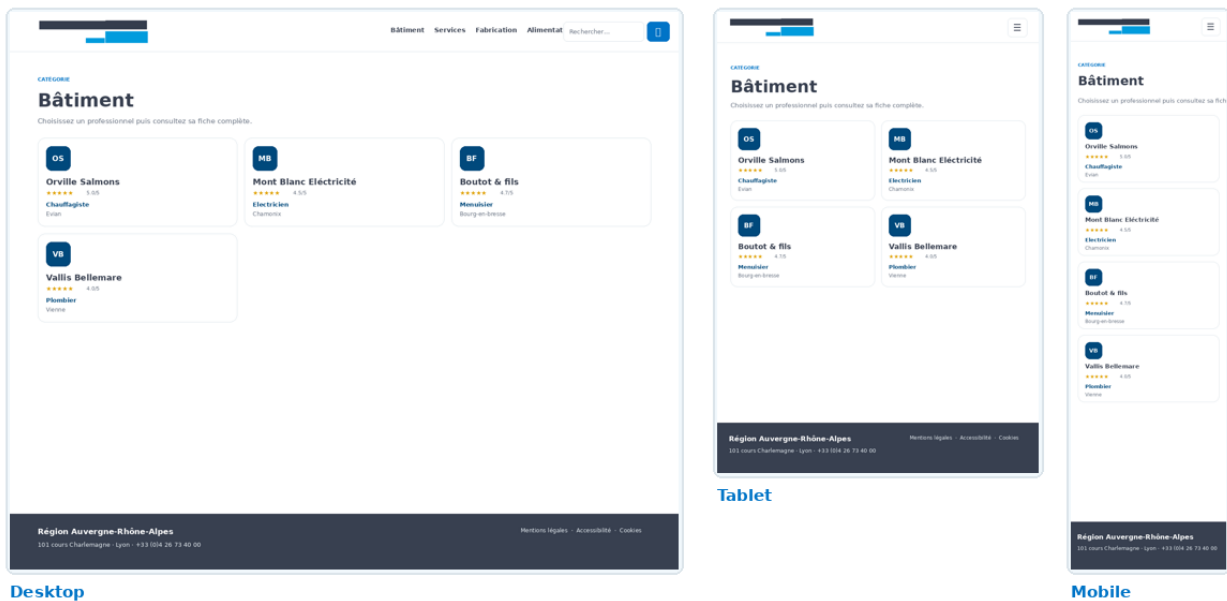


Figure 2 - Liste d'artisans : ordinateur, tablette et mobile.

Fiche artisan - déclinaisons responsive

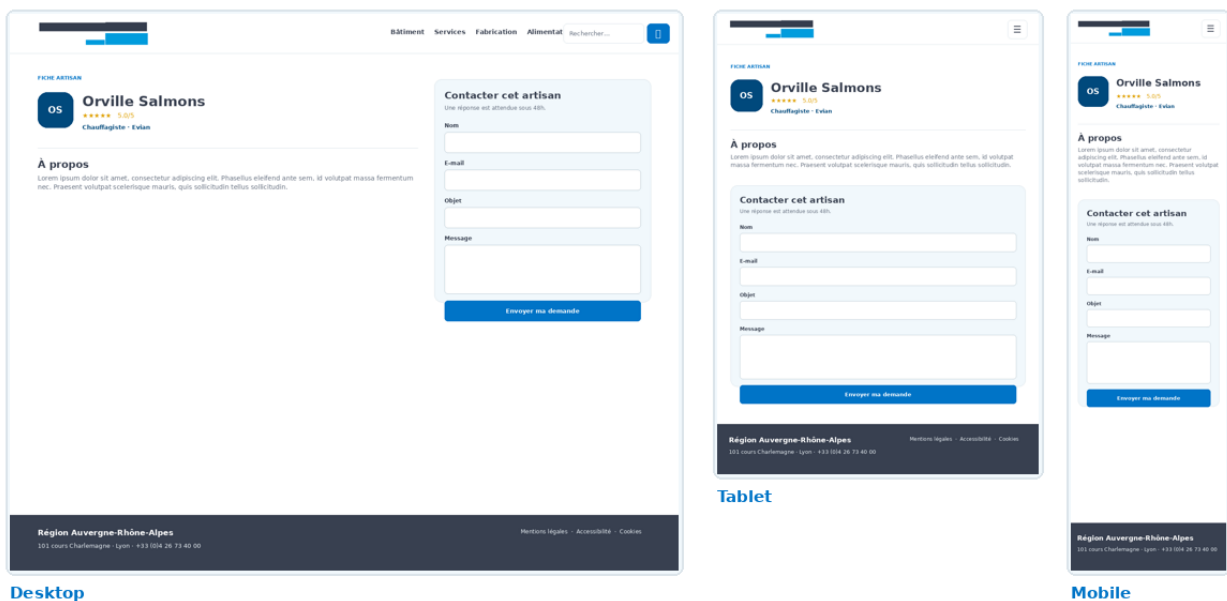


Figure 3 - Fiche artisan : ordinateur, tablette et mobile.

Pages légales et 404

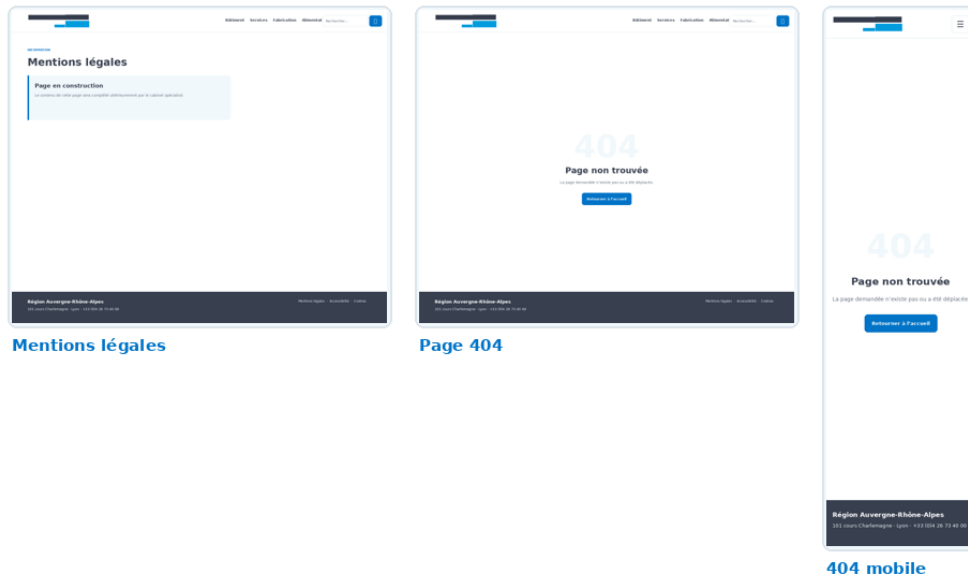


Figure 4 - Pages légales et page 404.

Lien Figma

Ouvrir les maquettes Figma

Maquettes Figma du projet - lien cliquable ci-dessus.

4. Architecture technique

4.1 Vue d'ensemble

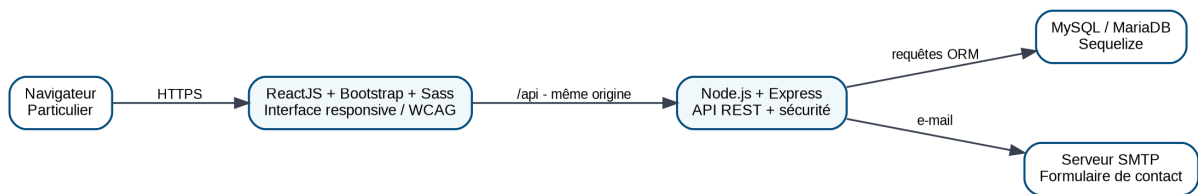


Figure 5 - Architecture de la solution.

Le navigateur charge l'application React. En développement, Vite redirige les appels /api vers Express. En production, Express peut servir le build React afin de conserver une même origine, ce qui simplifie la sécurité et le déploiement.

4.2 API REST

Méthode	Route	Rôle
GET	/api/categories	Liste des catégories
GET	/api/categories/:slug/artisans	Artisans d'une catégorie
GET	/api/artisans?search=nom	Recherche par nom
GET	/api/artisans/top	Trois artisans du mois
GET	/api/artisans/:id	Fiche publique d'un artisan
POST	/api/artisans/:id/contact	Envoi d'un message à l'artisan
GET	/api/health	Contrôle de disponibilité

4.3 Formulaire de contact

L'adresse e-mail de l'artisan n'est jamais renvoyée au navigateur. Le formulaire transmet le nom, l'e-mail du demandeur, l'objet et le message à l'API. Le serveur récupère ensuite l'adresse de l'artisan en base et envoie le message via SMTP.

5. Base de données - MCD et MLD

5.1 Choix de modélisation

Le schéma est normalisé autour de trois entités : categorie, specialite et artisan. Cette structure respecte directement les deux règles du brief : un artisan possède une seule spécialité et une spécialité appartient à une seule catégorie.

5.2 Modèle conceptuel de données (MCD)

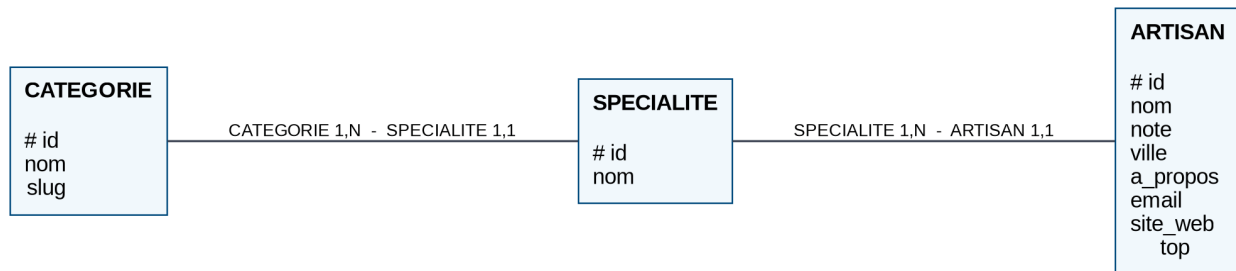


Figure 6 - MCD : cardinalités CATEGORIE 1,N -> SPECIALITE 1,1 puis SPECIALITE 1,N -> ARTISAN 1,1.

5.3 Modèle logique de données (MLD)

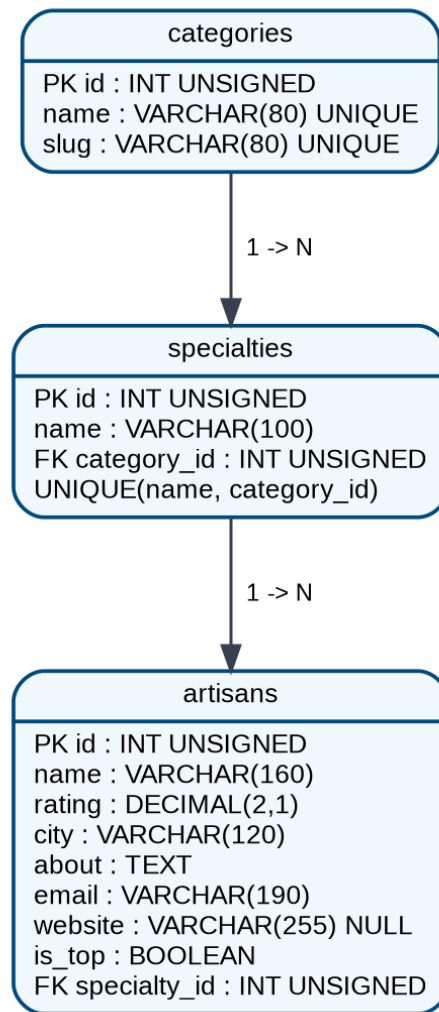


Figure 7 - MLD et clés étrangères.

5.4 Dictionnaire simplifié

Table	Champs principaux	Rôle
categories	id, name, slug	Catégories affichées dans le menu.
specialties	id, name, category_id	Métier rattaché à une catégorie.
artisans	id, name, rating, city, about, email, website, is_top, specialty_id	Données de la fiche artisan et sélection du mois.

Les scripts requis se trouvent dans database/01_create_database.sql et database/02_seed_database.sql. Le second reprend les 17 lignes du fichier data.xlsx fourni avec le sujet, sans corriger silencieusement les libellés ou e-mails.

6. Sécurité mise en place

Chaque mesure ci-dessous est implémentée dans le code et répond à un risque concret.

Mesure	Mise en œuvre	Intérêt
Helmet + CSP	Helmet ajoute des en-têtes HTTP de sécurité. Une Content Security Policy limite scripts, connexions, images, formulaires et intégration dans des frames.	Réduit l'impact de plusieurs classes de XSS, clickjacking et mauvaises configurations.
CORS restreint	Seules les origines déclarées dans FRONTEND_URL sont autorisées pour les requêtes navigateur.	Évite qu'un autre site web appelle librement l'API depuis le navigateur.
Même origine en production	Express peut servir le build React et l'API sous le même domaine.	Évite d'exposer un secret dans le JavaScript et réduit la surface cross-origine.
Validation des entrées	express-validator contrôle identifiants, slugs, recherche, e-mail, longueurs et contenu du formulaire.	Rejette les données malformées avant le traitement métier.
Sequelize	Les accès SQL utilisent l'ORM et non de la concaténation SQL brute.	Réduit le risque d'injection SQL.
E-mails masqués	Les contrôleurs excluent le champ email des réponses publiques.	Évite la collecte automatisée et la divulgation d'une donnée inutile au client.
Rate limiting	Limite globale et limite stricte de 5 formulaires / 15 min / IP.	Réduit le spam, l'abus et certaines attaques de déni de service.
Limite de taille	Corps JSON et URL-encoded limités à 20 Ko.	Évite des charges inutilement volumineuses.
Secrets hors Git	Mots de passe DB/SMTP dans server/.env, fichier ignoré par Git.	Évite la publication accidentelle de secrets.
Erreurs génériques	Les détails et stacks ne sont pas exposés en production.	Réduit les informations utiles à un attaquant.
URL artisans	Les sites externes doivent utiliser http ou https.	Évite des schémas de liens dangereux.
HTTPS de production	Le déploiement doit terminer les connexions en TLS/HTTPS.	Protège les données de formulaire en transit.

7. Veille sécurité et vulnérabilités

7.1 Sources suivies

- OWASP Top 10:2025 - <https://top10.owasp.org/2025/>
- OWASP SQL Injection Prevention Cheat Sheet - https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
- OWASP Cross Site Scripting Prevention Cheat Sheet - https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html
- OWASP Content Security Policy Cheat Sheet - https://cheatsheetseries.owasp.org/cheatsheets/Content_Security_Policy_Cheat_Sheet.html
- Express Production Security Best Practices - <https://expressjs.com/en/advanced/best-practice-security.html>
- Sequelize Validations & Constraints - <https://sequelize.org/docs/v6/core-concepts/validations-and-constraints/>

7.2 Points détectés pendant la revue

Point	Niveau	Correction
E-mail artisan exposé si le modèle complet est sérialisé	Élevé	Liste d'attributs publics ; email uniquement récupéré dans la route de contact.
Formulaire sans limitation de débit	Moyen	Ajout d'un rate limiter dédié + champ anti-bot.
CORS trop permissif	Moyen	Allowlist basée sur FRONTEND_URL.
Messages d'erreur trop détaillés	Faible à moyen	Réponse générique en production.
Risque d'injection par entrée utilisateur	Élevé	Validation + Sequelize ; aucune concaténation SQL brute.
Liens externes avec schéma non sûr	Moyen	Validation http/https uniquement.

7.3 Suivi des dépendances

Avant chaque livraison, exécuter `npm audit --omit=dev`, examiner les avis de sécurité et mettre à jour les dépendances compatibles. Le fichier `package-lock.json` doit être versionné lorsque l'installation npm est réalisée sur la machine de développement.

Limite technique importante

CORS ne constitue pas à lui seul un contrôle d'accès contre un client non navigateur. Pour une SPA publique, toute clé intégrée au JavaScript serait visible. Le choix défensif retenu consiste donc à déployer le front et l'API sous la même origine, à ne jamais exposer les identifiants MySQL et à placer l'application derrière le reverse proxy HTTPS de l'hébergeur.

8. Accessibilité, responsive et SEO

8.1 Accessibilité WCAG 2.1

- Lien d'évitement « Aller au contenu principal ».
- Navigation structurée avec header, nav, main, footer et titres hiérarchisés.
- Labels explicites sur les champs de recherche et de contact.
- Contrastes élevés avec la palette régionale.
- États de focus visibles grâce aux styles Bootstrap personnalisés.
- Alternative textuelle sur le logo.
- Note artisan lisible par lecteur d'écran (« Note x sur 5 ») en plus des étoiles visuelles.
- Prise en compte de `prefers-reduced-motion`.
- Mise en page mobile first et points de rupture 576, 768 et 992 px.

8.2 SEO

Le composant Seo gère un titre et une meta description pour l'accueil, les catégories, la recherche, la fiche artisan, les pages légales et la 404. Les titres restent descriptifs et centrés sur le contenu réel de la page.

8.3 Tests recommandés avant remise

- Valideur HTML du W3C.
- Lighthouse dans Chrome DevTools : Accessibilité, SEO et Bonnes pratiques.
- axe DevTools ou équivalent pour compléter l'audit WCAG.
- Tests clavier sans souris sur toutes les pages.
- Tests responsive aux largeurs 390 px, 768 px et 1440 px.

9. Repository, installation et scripts SQL

9.1 Contenu du dépôt

Le package fourni est structuré pour pouvoir être placé directement dans un repository GitHub.

- client/ : application React, composants, pages, Sass et assets.
- server/ : Express, routes, contrôleurs, modèles Sequelize, sécurité et service SMTP.
- database/01_create_database.sql : création du schéma.
- database/02_seed_database.sql : alimentation avec le fichier Excel fourni.
- docs/ : MCD, MLD, maquettes, veille sécurité et éléments de dossier.
- README.md : prérequis, installation, lancement et production.

Repository GitHub

Ouvrir le repository GitHub

9.2 Installation résumée

Étape	Commande / action
1	Installer Node.js 18+ et MySQL/MariaDB.
2	npm install à la racine.
3	mysql -u root -p < database/01_create_database.sql
4	mysql -u root -p < database/02_seed_database.sql
5	Copier server/.env.example vers server/.env et renseigner DB/SMTP.
6	npm run dev pour le développement.
7	npm run build puis NODE_ENV=production npm start.

10. Hébergement et mise en ligne

Le projet est prévu pour un hébergement Node.js avec une base MySQL/MariaDB. Le build React est servi par Express en production, ce qui permet un déploiement unique. L'hébergement doit fournir HTTPS et des variables d'environnement sécurisées.

10.1 Variables de production

- `NODE_ENV=production`
- `DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER`, `DB_PASSWORD`
- `FRONTEND_URL` avec l'URL publique
- `SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASS`, `SMTP_FROM`
- `TRUST_PROXY=1` si l'hébergeur termine HTTPS via un proxy

10.2 Étapes

- Créer la base MySQL de production et exécuter les deux scripts SQL.
- Renseigner les variables d'environnement sans les committer dans Git.
- Lancer le build React.
- Démarrer le serveur Node.js avec `NODE_ENV=production`.
- Vérifier `/api/health`, la recherche, les catégories, les fiches et le formulaire SMTP.
- Contrôler HTTPS, CORS et en-têtes de sécurité.

Site en ligne

Ouvrir le site en ligne

11. Conclusion et checklist de remise

Le dossier fourni couvre l'architecture du site, le code front-end et API, la base de données, la sécurité, la veille, le README, les scripts SQL et les maquettes de référence. Les seuls éléments Les liens Figma, GitHub et hébergement sont renseignés dans la version finale.

Checklist finale

Élément	État
Dossier PDF avec page de garde, sommaire, en-tête et pied-de-page	OK
Contexte, besoins, contraintes et livrables	OK
Captures de maquettes mobile/tablette/ordinateur	OK
Lien Figma	OK - lien ajouté
MCD	OK
MLD	OK
Sécurité détaillée	OK
Veille sécurité	OK
Repository GitHub avec code	OK - publié
Script SQL création	OK
Script SQL alimentation	OK
README.md	OK
Site accessible en ligne	À déployer

Prochaine action

Remplacer les trois balises de lien dans ce PDF, dans README.md et dans

Les liens Figma, GitHub et hébergement sont renseignés dans la version finale.